

FIAP ON

06

IA CRIATIVA: EXPLORANDO O UNIVERSO DOS MODELOS GERATIVOS PROFUNDOS

PDF exclusivo para Murilo Salla - rm568041
murilosalla@protonmail.com

LISTA DE FIGURAS

Figura 1 - Shallow *versus* Deep Neural Networks 8
Figura 2 - Exemplos de imagens geradas por GANs 9
Figura 3 - Diagrama comparativo de AEs, VAEs, GANs e Diffusion Models 10
Figura 4 - Arquitetura Encoder-Decoder 11
Figura 5 - Arquitetura visual de um autoencoder aplicado ao dataset MNIST 12
Figura 6 - Visualização do espaço latente com t-SNE mostrando agrupamentos de dígitos MNIST 14
Figura 7 - Denoising Autoencoders 16
Figura 8 - Arquitetura completa de um VAE 20
Figura 8 - Arquitetura completa de uma GAN 25
Figura 10 - Convergência da GAN 27
Figura 11 - Sequência de imagens geradas durante o treinamento da GAN 28
Figura 12 - Deep Convolution GAN 32
Figura 13 - CycleGAN. 33
Figura 14 - StyleGAN 34
Figura 15 - Imagens geradas usando VAE (acima) e GAN (abaixo) 37
Figura 16 - Imagens geradas utilizando VAE 44
Figura 17 - Sequência de imagens interpoladas entre dois dígitos, mostrando transição visual suave 45
Figura 18 - Imagens geradas utilizando GAN 48

FIAP

LISTA DE QUADROS

Quadro 1 - Comparação entre DCGAN, CycleGAN e StyleGAN 35
Quadro 2 - Comparação entre VAE e GAN.....39

FIAP

SUMÁRIO

1 IA CRIATIVA: EXPLORANDO O UNIVERSO DOS MODELOS GERATIVOS PROFUNDOS 6
1.1 Introdução 6
1.2 O que são modelos generativos? 6
1.3 Por que “Deep”? O papel das redes neurais 7
1.4 Aplicações práticas dos modelos generativos 8
1.5 Tipos de modelos generativos profundos (visão geral) 9

2 AUTOENCODERS (AE) 11
2.1 Arquitetura de um autoencoder 11
2.2 Função de Perda (Loss Function): Erro Quadrático Médio (MSE) 12
2.3 Espaço latente (Latent Space) e representações 13
2.4 Autoencoders undercomplete *versus* overcomplete 14
2.5 Aplicações práticas dos autoencoders 15
2.6 Intuição Matemática: Forward Pass e Erro de Reconstrução 16
2.7 Conclusão 16

3 VARIATIONAL AUTOENCODERS (VAE) 18
3.1 Limitações dos autoencoders tradicionais 18
3.2 Espaço latente como distribuição 19
3.3 Codificador e decodificador probabilísticos 19
3.4 Conceitos matemáticos fundamentais 20
3.5 Truque da reparametrização 21
3.6 Exemplo prático: Geração de dígitos com o MNIST 21
3.7 Vantagens e desvantagens dos VAEs 21
3.8 Exemplo de Código (PyTorch): VAE Simples 22

4 REDES GERADORAS ADVERSARIAIS (GANS) 24
4.1 Intuição das GANs: Duelo entre dois modelos 24
4.2 Gerador *versus* discriminador 25
4.3 Função objetivo: Minimax Game 26
4.4 Teoria Matemática: Divergência de Jensen-Shannon 26
4.5 Por que as GANs são difíceis de treinar? 27
4.6 Exemplos visuais: Real *versus* Fake e caminhada latente 28
4.7 Exemplo de código (PyTorch): DCGAN com CIFAR-10 28
4.8 Dicas de treinamento para GANs 30
4.9 Conclusão 30

5 EXTENSÕES E VARIAÇÕES DAS GANS 31
5.1 DCGAN: Redes Convolucionais como Base para GANs Estáveis 31
5.2 CycleGAN: Tradução de Domínios sem Pares (Unpaired Image-to-Image Translation) 32
5.3 StyleGAN: Geração de Rostos Realistas e Controle de Estilo 33
5.4 Comparação entre DCGAN, CycleGAN e StyleGAN 34
5.5 Conclusão 35

6 VAES *VERSUS* GANS: UMA ANÁLISE COMPARATIVA 36
6.1 Qualidade das Saídas: Realismo *versus* Coerência Estatística 36
6.2 Estrutura do Espaço Latente: Regularidade *versus* Complexidade 37
6.3 Estabilidade no Treinamento: Convergência *versus* Instabilidade 38

6.4 Comparação de Aplicações: Quando usar cada um 38
6.5 Conclusão 39

7 TUTORIAL PRÁTICO: IMPLEMENTANDO UM VAE E UMA GAN COM MNIST ..41
7.1 Preparação: Imports e Dataset.....41
7.2 VAE: Construindo o encoder e o decoder 42
7.2.1 Função de perda e treinamento do VAE 43
7.2.2 Gerando novas amostras com o VAE 43
7.2.3 Explorando o espaço latente com o VAE 44
7.3 Treinando uma GAN simples com MNIST.....45
7.3.1 Treinamento da GAN.....46
7.3.2 Gerando imagens com a GAN 47
7.4 Dicas de avaliação 48

CONCLUSÃO.....49

REFERÊNCIAS.....50

FIAP

1 IA CRIATIVA: EXPLORANDO O UNIVERSO DOS MODELOS GERATIVOS PROFUNDOS

1.1 Introdução

Os modelos generativos representam uma das áreas mais fascinantes e transformadoras da Inteligência Artificial moderna. Diferentemente dos modelos discriminativos, que aprendem a classificar ou prever uma saída a partir de uma entrada, como dizer se uma imagem contém um gato ou um cachorro, os modelos generativos têm o objetivo de aprender a distribuição dos dados e gerar novos exemplos plausíveis que se assemelham aos dados originais. Em outras palavras, enquanto um modelo discriminativo responde a perguntas do tipo “o que é isso?”, um modelo generativo busca responder “como criar algo semelhante a isso?”.

Para entender melhor, podemos pensar em um exemplo do cotidiano. Imagine que você tem milhares de imagens de rostos humanos reais. Um modelo generativo bem treinado, como o StyleGAN (Karras *et al*., 2019), é capaz de criar imagens de rostos que não pertencem a nenhuma pessoa real, mas que parecem extremamente convincentes. O resultado é tão realista que, muitas vezes, é impossível distinguir a imagem criada pelo modelo de uma foto verdadeira. Essas tecnologias já estão sendo utilizadas, por exemplo, na criação de personagens de videogames, em efeitos especiais para cinema e até em bancos de imagens que não violam direitos autorais, porque os rostos são totalmente sintéticos.

1.2 O que são modelos generativos?

Matematicamente, um modelo generativo tenta aprender a função de probabilidade conjunta $P(X)$ ou $P(X, Y)$, dependendo do contexto, onde X representa os dados de entrada. Por exemplo, se temos um conjunto de imagens de roupas (como o famoso dataset FashionMNIST), o modelo busca compreender a estrutura e os padrões internos dessas imagens: texturas, formas e cores. Depois de aprender esses padrões, ele consegue gerar novas imagens de roupas que nunca existiram, mas que mantêm a coerência com o conjunto de treinamento.

Um exemplo clássico é o GPT (Generative Pre-trained Transformer), que aprendeu padrões de linguagem humana lendo bilhões de palavras. Atualmente, ele é capaz de gerar textos completos, criar diálogos, escrever código e até compor músicas, pois internalizou as regras estatísticas e semânticas da língua. Da mesma forma, os modelos de difusão como o Stable Diffusion conseguem criar imagens a partir de descrições textuais, como “um castelo futurista em Marte pintado no estilo Van Gogh”.

1.3 Por que “Deep”? O papel das redes neurais

Antes da revolução do Aprendizado Profundo (Deep Learning), os modelos generativos dependiam de técnicas mais simples, como Gaussian Mixture Models (GMM) ou Hidden Markov Models (HMM). Embora úteis, esses métodos tinham dificuldades para lidar com dados complexos, como imagens de alta resolução ou texto natural.

Com o avanço das redes neurais profundas, tornou-se possível representar padrões muito mais complexos. As redes profundas atuam como poderosos aproximadores universais, capazes de aprender funções altamente não lineares. Um modelo generativo profundo combina a ideia de aprender distribuições de dados com a capacidade das redes neurais de extrair representações hierárquicas.

Por exemplo, em uma Rede Neural Convolucional (CNN) utilizada dentro de um Generative Adversarial Network (GAN), os primeiros filtros podem aprender bordas e formas básicas, enquanto as camadas mais profundas capturam detalhes de textura, iluminação e até expressões faciais. Esse aprendizado hierárquico é a chave para a qualidade e o realismo das amostras geradas.

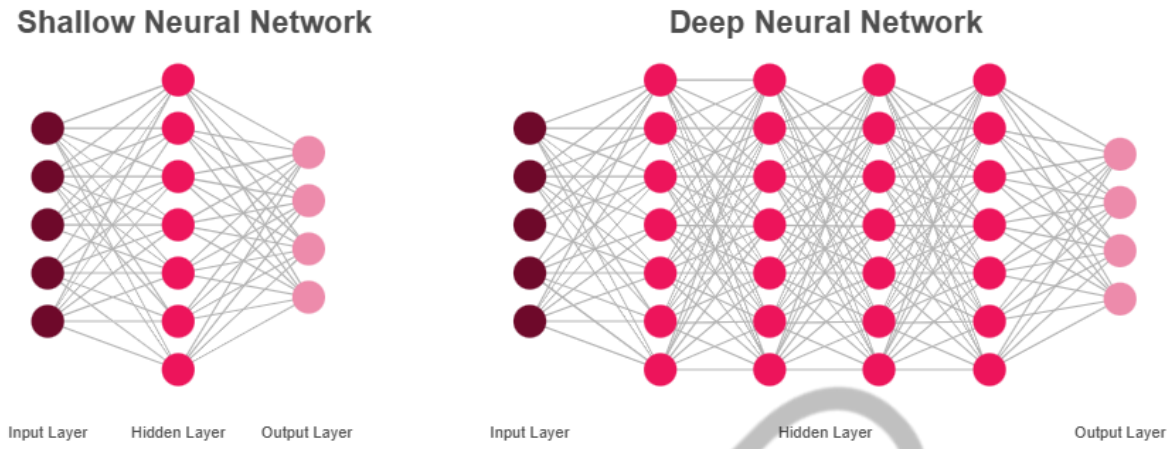


Figura 1 - Shallow *versus* Deep Neural Networks
Fonte: Medium (2024), adaptado por FIAP (2026)

1.4 Aplicações práticas dos modelos generativos

Os modelos generativos já estão presentes em uma série de aplicações do mundo real, muitas vezes sem que percebamos. Síntese de imagens é talvez a aplicação mais famosa, com ferramentas como o DALL·E e o Midjourney capazes de criar imagens a partir de comandos em linguagem natural, transformando texto em arte visual.

Na área de texto, os Large Language Models (LLMs), como o ChatGPT, não apenas respondem perguntas, mas também podem redigir redações, criar histórias, gerar resumos e até escrever código. Em áudio, modelos como o Jukebox da OpenAI podem compor músicas originais em estilos variados.

Outro campo promissor é a aumentação de dados (*data augmentation*). Em cenários onde temos poucos exemplos reais para treinar um modelo (por exemplo, imagens médicas raras, como tumores específicos), modelos generativos podem criar novos exemplos sintéticos que ajudam a melhorar a performance de algoritmos de diagnóstico. Essa abordagem é estudada, por exemplo, em trabalhos como Frid-Adar *et al*. (2018), que exploraram GANs para gerar imagens médicas sintéticas para treinamento de redes de classificação.

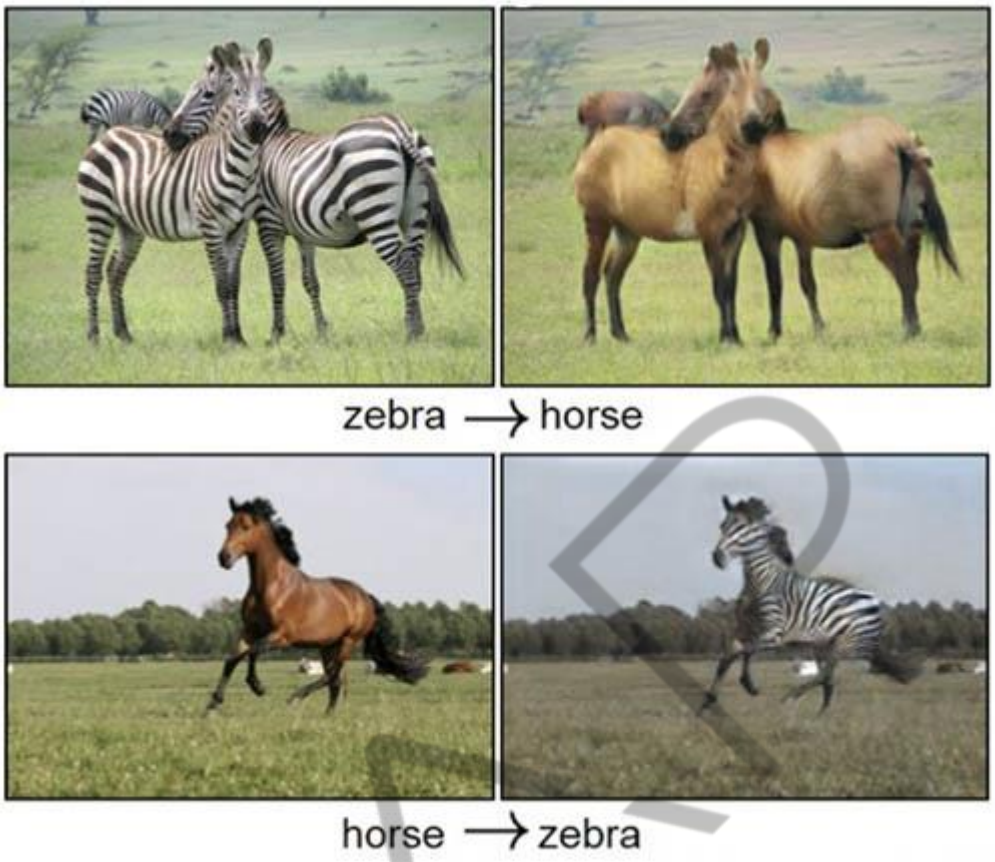


Figura 2 - Exemplos de imagens geradas por GANs
Fonte: AWS (2025)

1.5 Tipos de modelos generativos profundos (visão geral)

Dentro do universo dos modelos generativos profundos, existem diferentes arquiteturas, cada uma com suas características e vantagens:

- **Autoencoders (AEs):** são modelos que aprendem a comprimir e reconstruir os dados. Embora não sejam “gerativos” em sua forma mais pura, eles servem como base para modelos mais avançados como os Variational Autoencoders (VAEs).
- **Variational Autoencoders (VAEs):** introduzem uma interpretação probabilística ao espaço latente, permitindo gerar novas amostras ao “navegar” em um espaço de distribuições.
- **Generative Adversarial Networks (GANs):** baseiam-se em um jogo de dois jogadores, um gerador que cria exemplos falsos e um discriminador

que tenta detectar se um exemplo é real ou falso. O treinamento resulta em amostras altamente realistas.

- **Modelos de Difusão (Diffusion Models):** como o Stable Diffusion, que gradualmente "desfazem" o ruído em uma imagem para gerar conteúdos coerentes a partir de uma descrição textual.

Este capítulo será dedicado especialmente aos **VAEs** e **GANs**, pois eles representam marcos fundamentais na evolução dos modelos generativos profundos. Ao longo das próximas seções, vamos explorar sua matemática, arquitetura e aplicações com exemplos práticos em código.

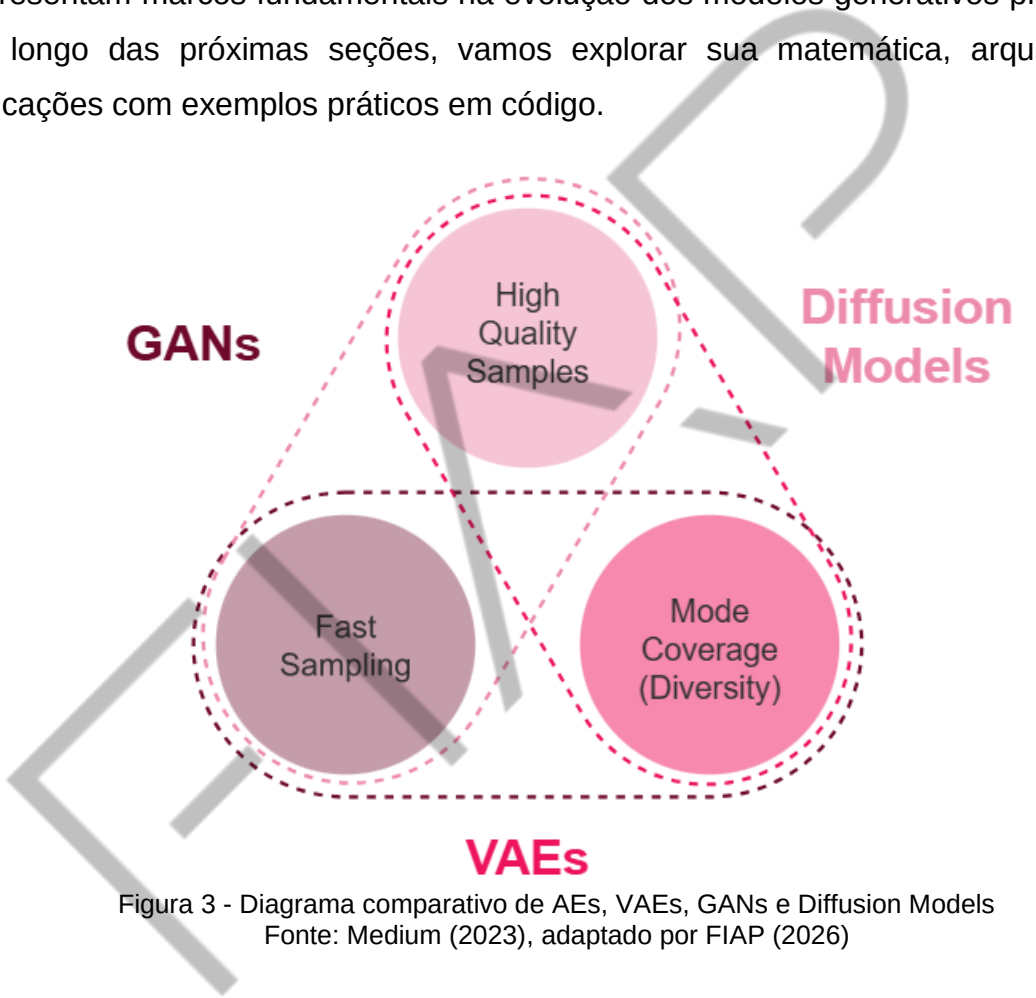


Figura 3 - Diagrama comparativo de AEs, VAEs, GANs e Diffusion Models
Fonte: Medium (2023), adaptado por FIAP (2026)

2 AUTOENCODERS (AE)

Para começarmos a explorar o mundo dos modelos generativos profundos, vamos entender uma arquitetura fundamental: os autoencoders. Embora nem sempre sejam considerados modelos gerativos no sentido estrito — já que sua função principal é reconstruir a entrada em vez de criar algo totalmente novo —, os autoencoders são a base conceitual para modelos mais sofisticados como os Variational Autoencoders (VAEs).

Os autoencoders funcionam como uma espécie de máquina de compressão e descompressão de dados. Eles são compostos por duas partes principais: o **codificador** (encoder) e o **decodificador** (decoder).

O encoder recebe uma entrada x (por exemplo, uma imagem), e a transforma em uma representação compacta chamada de vetor latente ou representação latente, denotado por z. Esse vetor contém as informações mais essenciais da entrada original, mas em uma forma comprimida e abstrata. Em seguida, o decoder tenta reconstruir a entrada original a partir de z produzindo uma saída $\hat{x}$. O objetivo é que essa reconstrução seja a mais fiel possível à entrada original.

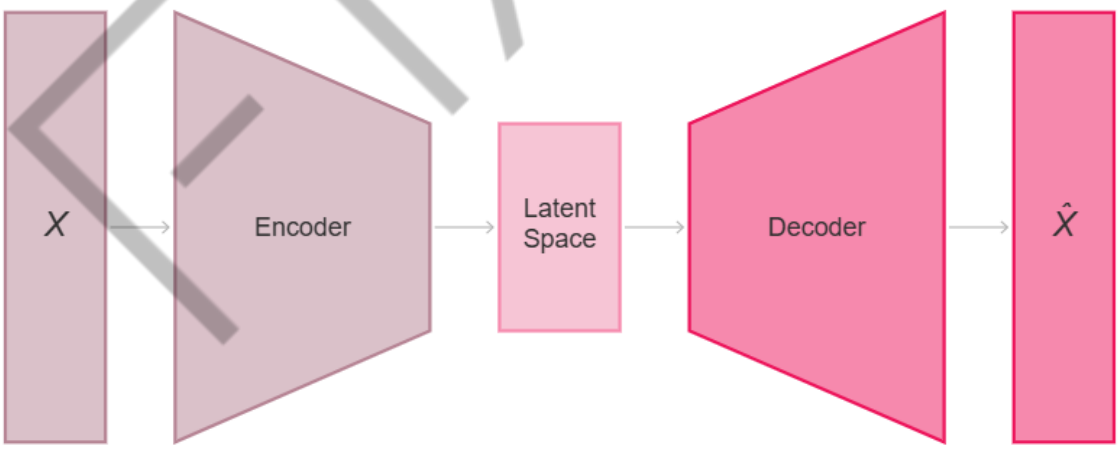


Figura 4 - Arquitetura Encoder-Decoder
Fonte: ResearchGate (2023), adaptado por FIAP (2026)

2.1 Arquitetura de um autoencoder

Na prática, tanto o encoder quanto o decoder são redes neurais, geralmente multicamadas, que aprendem funções parametrizadas. O encoder aprende uma

função $f_\theta(x) = z$ que projeta a entrada para o espaço latente, enquanto o decoder aprende uma função $g_\phi(z) = \hat{x}$, que tenta reconstruir os dados originais.

Se estivermos lidando com imagens, é comum que o encoder use **camadas convolucionais** (CNNs) para capturar padrões espaciais como bordas, texturas e formas. Já o decoder pode utilizar camadas de **transposição convolucional** (conhecidas como deconvoluções) ou ***upsampling*** para aumentar a resolução da imagem a partir do vetor comprimido.

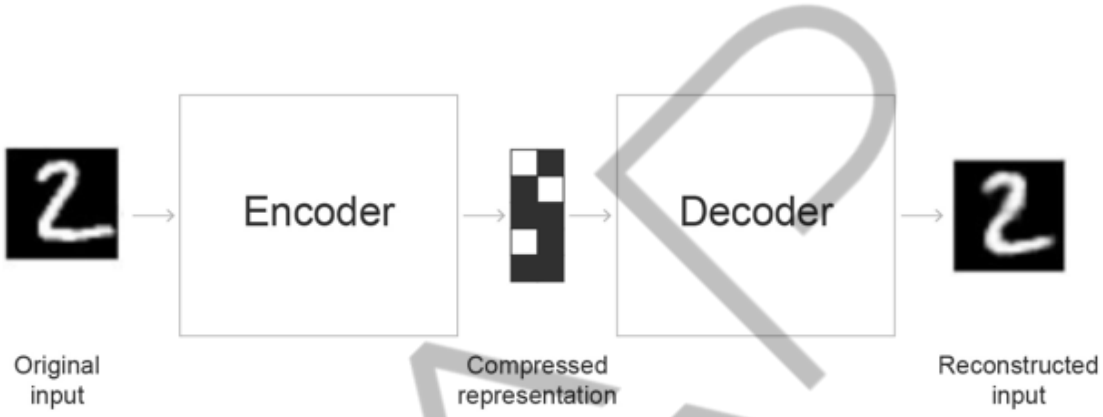


Figura 5 - Arquitetura visual de um autoencoder aplicado ao dataset MNIST
Fonte: Medium (2020), adaptado por FIAP (2026)

2.2 Função de Perda (Loss Function): Erro Quadrático Médio (MSE)

Durante o treinamento, o autoencoder ajusta seus pesos de forma que a saída reconstruída $\hat{x}$ fique o mais próxima possível da entrada original x. Para medir essa diferença, é comum utilizarmos a função de perda conhecida como **Erro Quadrático Médio (MSE – Mean Squared Error)**:

$$\mathcal{L}(x, \hat{x}) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{x}_i)^2$$

Esse valor é minimizado com o uso de algoritmos de otimização como o **gradient descent**, com retropropagação do erro (backpropagation) para atualizar os pesos do encoder e do decoder.

Do ponto de vista matemático, o treinamento do autoencoder é uma forma de **aprendizado não supervisionado**, pois o modelo está sendo treinado para mapear os dados em si, sem precisar de rótulos externos.

2.3 Espaço latente (Latent Space) e representações

A parte mais interessante dos autoencoders talvez seja o espaço latente. Ao comprimir a informação em vetores de dimensão reduzida (por exemplo, de uma imagem de 784 pixels para um vetor de 16 dimensões), o modelo é forçado a aprender as características mais relevantes dos dados.

Esse espaço latente pode ser explorado de várias formas. Em um autoencoder treinado com imagens de dígitos manuscritos, por exemplo, é possível visualizar os vetores latentes com técnicas de projeção como PCA ou t-SNE e observar como diferentes dígitos se agrupam nesse espaço abstrato. Isso mostra que o modelo aprendeu a organizar semanticamente os dados, mesmo sem supervisão explícita.

Além disso, ao manipular os vetores z diretamente, podemos criar combinações entre diferentes exemplos. Por exemplo, ao interpolar entre dois vetores latentes de imagens de “3” e “8”, o decoder pode gerar imagens que gradualmente transitam entre esses dois números, revelando uma estrutura contínua e semântica no espaço latente.

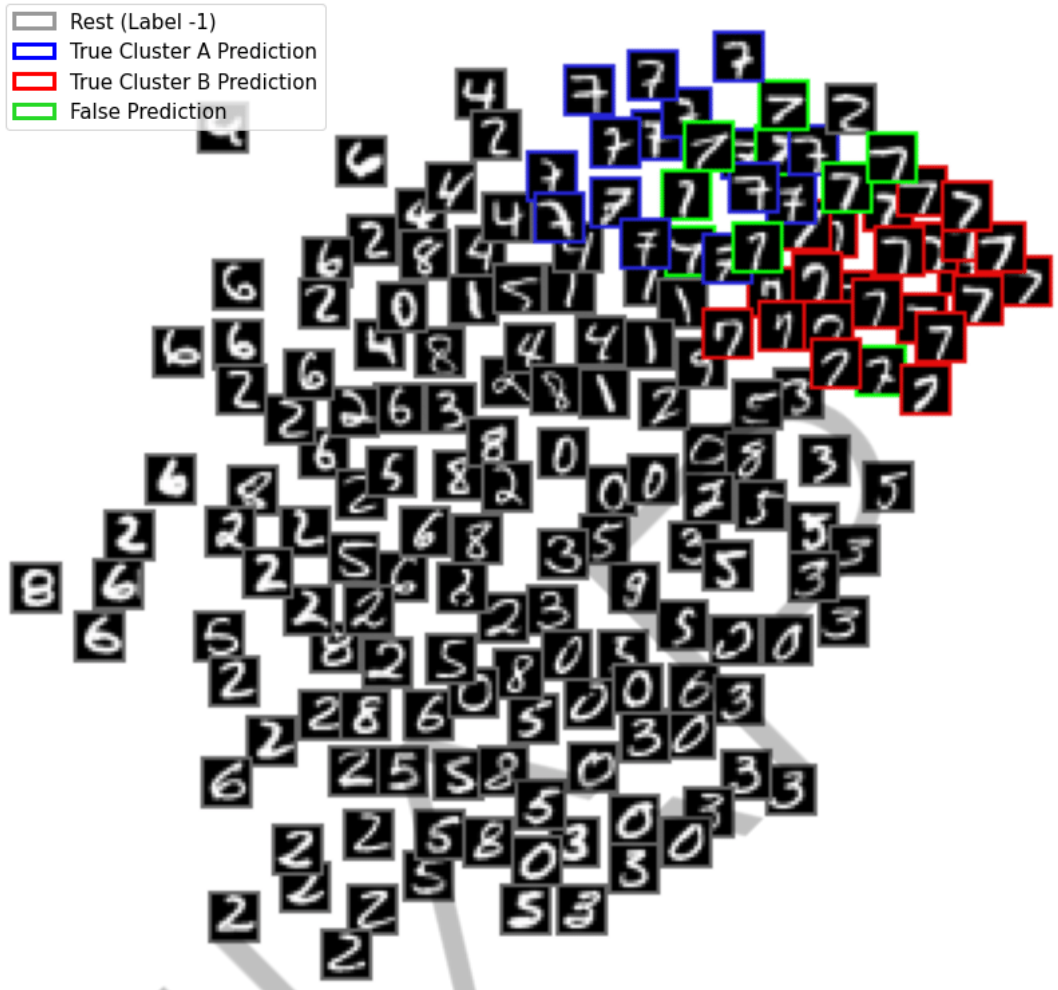


Figura 6 - Visualização do espaço latente com t-SNE mostrando agrupamentos de dígitos MNIST
Fonte: UCSL (2021)

2.4 Autoencoders undercomplete *versus* overcomplete

Existem dois tipos principais de autoencoders: os **undercomplete** e os **overcomplete**. A diferença está na dimensão do vetor latente em relação à dimensão da entrada.

Um **autoencoder undercomplete** possui um espaço latente de menor dimensão que a entrada. Essa restrição força o modelo a comprimir as informações e, portanto, aprender representações significativas. Essa é a abordagem mais comum quando o objetivo é reduzir dimensionalidade ou aprender embeddings.

Por outro lado, um **autoencoder overcomplete** possui um espaço latente de dimensão igual ou maior que a entrada. Se não houver outras restrições, como regularizações ou ruído, o modelo pode simplesmente aprender a copiar a entrada para a saída sem aprender padrões úteis. Por isso, em modelos overcomplete, é

comum utilizar técnicas como sparsidade ou autoencoders com ruído (denoising autoencoders) para evitar esse comportamento trivial.

2.5 Aplicações práticas dos autoencoders

Os autoencoders têm uma gama diversa de aplicações na indústria e na pesquisa. Uma das mais conhecidas é a redução de dimensionalidade, onde substituem métodos tradicionais como o PCA com a vantagem de capturar relações não lineares entre os dados. Em sistemas de recomendação, por exemplo, os autoencoders podem ser utilizados para aprender embeddings de usuários e produtos.

Outra aplicação importante é na remoção de ruído (denoising). Ao treinar um autoencoder para reconstruir uma imagem original a partir de uma versão com ruído adicionado, o modelo aprende a remover esse ruído automaticamente. Essa técnica é usada em processamento de imagens médicas, restauração de fotos antigas e filtros de imagem em smartphones.

Na área de detecção de anomalias, os autoencoders são extremamente eficazes. Como aprendem a reconstruir bem os exemplos “normais”, qualquer entrada cuja reconstrução resulte em um erro alto é provavelmente uma anomalia. Isso é útil para a detecção de fraudes em transações financeiras, identificação de falhas em sensores industriais e o monitoramento de saúde de máquinas.

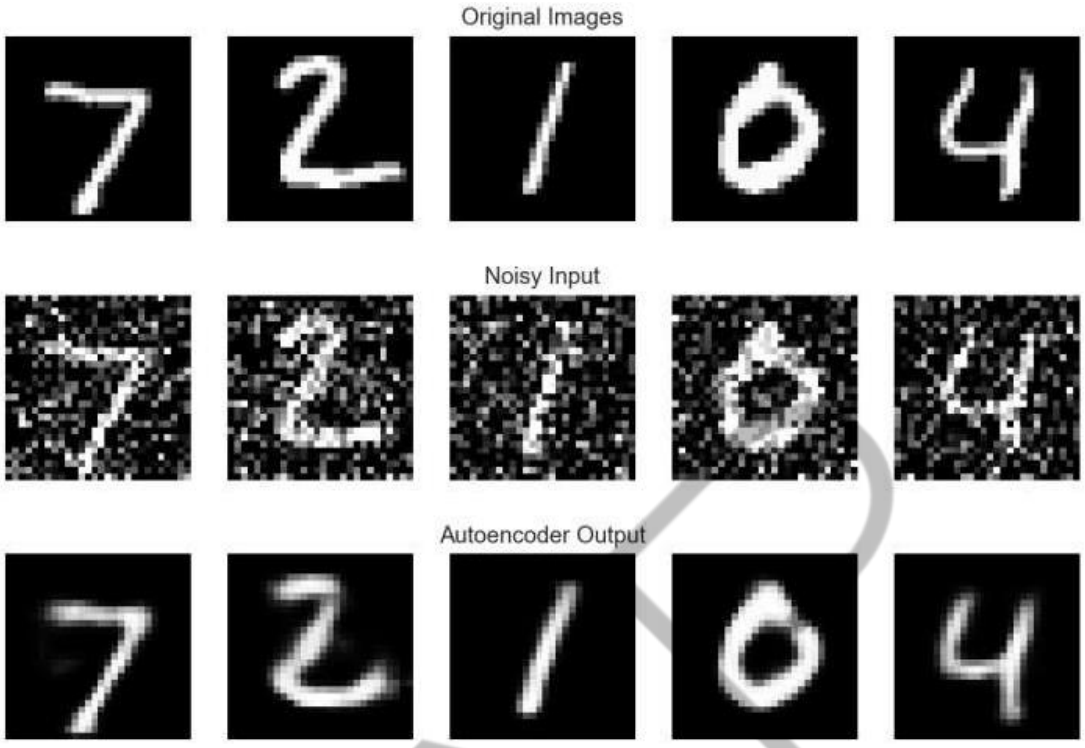


Figura 7 - Denoising Autoencoders
Fonte: Medium (2019)

2.6 Intuição Matemática: Forward Pass e Erro de Reconstrução

Vamos considerar um exemplo simples: imagine uma imagem de 28x28 pixels (como no MNIST), que é achatada em um vetor de 784 elementos. O encoder aplica uma sequência de funções não lineares (camadas com ReLU, por exemplo) até reduzir esse vetor para, digamos, 32 dimensões. Esse é o nosso vetor latente z.

O decoder então faz o caminho inverso, aplicando funções que aumentam gradualmente a dimensão do vetor até retornar aos 784 pixels originais. O modelo compara essa saída com a entrada original e calcula o MSE. O processo de **forward pass** é simplesmente o fluxo da entrada até a saída, passando pelo espaço latente. Já o **backward pass** atualiza os pesos para minimizar o erro de reconstrução, tornando a rede melhor a cada época de treinamento.

2.7 Conclusão

Os autoencoders representam um passo fundamental para quem deseja entender como as máquinas podem aprender estruturas internas dos dados de forma

não supervisionada. Sua simplicidade arquitetural esconde uma enorme potência, especialmente quando explorada em variações como os **autoencoders variacionais**. Compreender a dinâmica entre encoder, decoder e espaço latente é essencial para avançarmos para modelos verdadeiramente generativos, capazes não apenas de reconstruir, mas de criar. Nas próximas seções, mergulharemos nessa capacidade com o estudo dos **VAEs** e dos **GANs**, onde a criatividade computacional se manifesta em sua forma mais impressionante.

FIAP

3 VARIATIONAL AUTOENCODERS (VAE)

Os autoencoders variacionais — conhecidos como VAEs — representam uma evolução conceitual e matemática importante dos autoencoders tradicionais. Embora os autoencoders simples (também chamados de *vanilla autoencoders*) sejam úteis para compressão, reconstrução e até redução de dimensionalidade, eles não são propriamente modelos generativos no sentido pleno da palavra. Isso ocorre porque os autoencoders tradicionais não aprendem a distribuição dos dados de forma explícita. Eles simplesmente aprendem a mapear um dado de entrada para um espaço latente e depois a retornar ao domínio original. No entanto, o espaço latente aprendido por eles é muitas vezes desestruturado, desconexo e não contínuo — ou seja, não é possível amostrar um vetor qualquer do espaço latente e gerar uma saída válida.

É justamente esse problema que os VAEs buscam resolver. Introduzidos por Kingma e Welling (2013) no artigo *Auto-Encoding Variational Bayes*, os VAEs incorporam conceitos da estatística bayesiana à arquitetura do autoencoder. O resultado é um modelo que, além de comprimir e reconstruir dados, aprende uma distribuição probabilística sobre o espaço latente, permitindo gerar novas amostras de forma coerente e contínua.

3.1 Limitações dos autoencoders tradicionais

O principal problema dos autoencoders comuns é que eles não controlam a organização do espaço latente. O vetor z resultante do encoder pode assumir valores muito dispersos, formando ilhas desconectadas no espaço latente. Com isso, se tentarmos amostrar um vetor z aleatório (por exemplo, de uma distribuição normal) e passá-lo ao decoder, a probabilidade de obter uma amostra realista é muito baixa. Além disso, o espaço latente não tem propriedades úteis, como continuidade e interpolação suave entre diferentes exemplos.

Os VAEs superam essas limitações ao forçar o espaço latente a seguir uma distribuição conhecida, geralmente uma distribuição normal multivariada N(0,I). Isso torna possível navegar no espaço latente, interpolar entre exemplos e gerar novas amostras simplesmente sorteando vetores z dessa distribuição padrão.

3.2 Espaço latente como distribuição

Nos VAEs, o vetor latente z não é mais um vetor fixo de valores determinísticos. Em vez disso, o encoder aprende dois vetores: a média μ e o desvio padrão σ, que definem uma distribuição gaussiana multivariada para cada entrada. Assim, ao invés de dizer “essa imagem vira o vetor z”, o modelo diz: “essa imagem é representada por uma distribuição $N(\mu, \sigma^2)$”.

Durante o treinamento, o vetor z é amostrado a partir dessa distribuição utilizando uma técnica chamada truque de reparametrização (que veremos adiante). Isso garante que o espaço latente tenha uma estrutura contínua, regularizada e compatível com uma distribuição de probabilidade conhecida.

3.3 Codificador e decodificador probabilísticos

No VAE, o encoder é uma rede neural que não retorna diretamente o vetor latente, mas sim dois vetores μ (média) e $log(\sigma^2)$ (variância logarítmica). Esses vetores definem a distribuição latente $q(z|x)$ para cada dado de entrada x.

O decoder, por sua vez, também é probabilístico: ele aprende a mapear o vetor amostrado z de volta para o espaço de dados, gerando uma distribuição $p(z|x)$. Assim, o processo todo se transforma em uma cadeia probabilística: do dado x, extraímos uma distribuição de z, da qual amostramos e, a partir disso, geramos uma nova x.

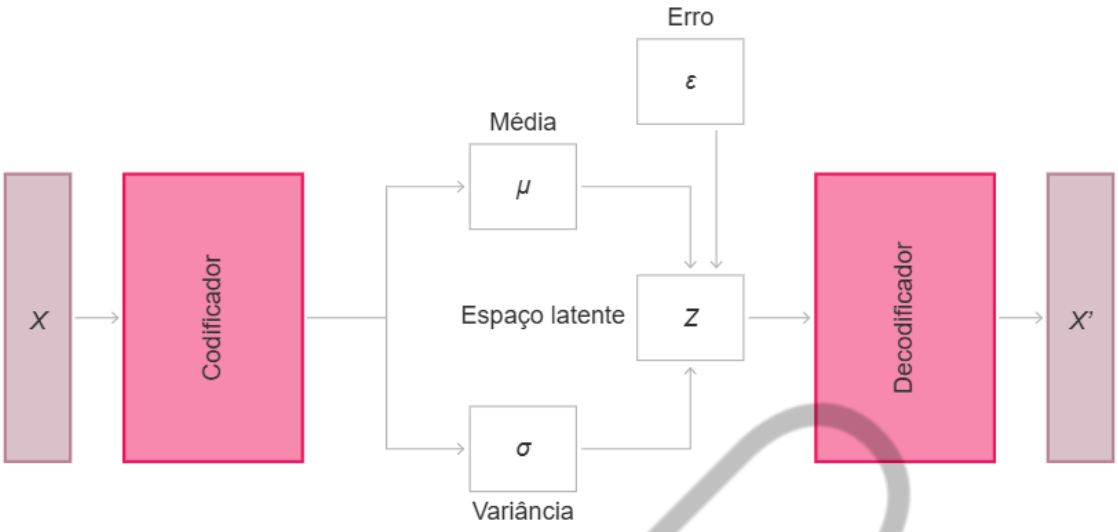


Figura 8 - Arquitetura completa de um VAE
Fonte: ResearchGate (2022), adaptado por FIAP (2026)

3.4 Conceitos matemáticos fundamentais

Para treinar o VAE, não utilizamos apenas o erro de reconstrução como no AE tradicional. É necessário considerar também o quanto a distribuição aprendida $q(z|x)$ se afasta da distribuição desejada $p(z) = N(0, I)$. Esse afastamento é medido por uma função chamada divergência de Kullback-Leibler (KL divergence).

A função de perda do VAE, chamada de Evidence Lower Bound (ELBO), combina dois termos:

$$\mathcal{L}(x) = E_{q(z|x)}[\log \mathrm{p}(x|z)] - D_{KL}\big(q(z|x)\big)||p(z)$$

O primeiro termo representa a **verossimilhança** da reconstrução (semelhante ao MSE). O segundo termo é a **penalização** pelo afastamento da distribuição latente em relação à distribuição padrão. O objetivo é maximizar essa função (ou minimizar o seu negativo), o que significa gerar boas reconstruções e manter o espaço latente regularizado.

A divergência KL entre duas distribuições normais multivariadas possui uma forma fechada simples, o que facilita sua implementação.

3.5 Truque da reparametrização

Um desafio técnico importante é que, para calcular o gradiente da função de perda, é necessário retropropagar o erro através da amostragem do vetor z, o que não é possível diretamente (amostrar não é uma operação diferenciável). A solução proposta por Kingma e Welling foi o truque da reparametrização, que consiste em reescrever o vetor z da seguinte forma: $z = \mu + \sigma \odot \epsilon$ com $\epsilon \sim N(0, I)$.

Dessa forma, em vez de amostrar diretamente de $N(\mu, \sigma^2)$, o modelo amostra de uma normal padrão ϵ, e usa operações diferenciáveis para gerar z. Isso permite que o erro seja retropropagado normalmente.

3.6 Exemplo prático: Geração de dígitos com o MNIST

Um dos exemplos mais clássicos de aplicação de VAEs é a geração de dígitos manuscritos utilizando o dataset MNIST. Após treinar um VAE com imagens de dígitos de 0 a 9, o modelo aprende a estruturar o espaço latente de forma que regiões distintas correspondam a diferentes classes. Ao amostrar vetores zzz desse espaço, é possível gerar imagens de dígitos nunca antes vistos, mas coerentes com o estilo do conjunto de treinamento.

3.7 Vantagens e desvantagens dos VAEs

Os VAEs têm uma série de vantagens. Sua capacidade de navegar no espaço latente de forma contínua, a facilidade de amostragem direta e a interpretação probabilística sólida fazem deles uma escolha robusta em diversos cenários. Além disso, assim como a função de perda é baseada em uma formulação variacional bem definida, o treinamento é mais estável que em GANs.

Por outro lado, os VAEs muitas vezes produzem amostras borradas ou menos realistas do que as geradas por GANs. Isso ocorre porque o modelo maximiza uma função de verossimilhança que, ao tentar cobrir toda a distribuição dos dados, pode suavizar os detalhes nas amostras.

3.8 Exemplo de Código (PyTorch): VAE Simples

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class Encoder(nn.Module):
      def    __init__(self,    input_dim=784,    hidden_dim=400,
latent_dim=20):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc_mu = nn.Linear(hidden_dim, latent_dim)
        self.fc_logvar = nn.Linear(hidden_dim, latent_dim)

    def forward(self, x):
        h = F.relu(self.fc1(x))
        mu = self.fc_mu(h)
        logvar = self.fc_logvar(h)
        return mu, logvar

class Decoder(nn.Module):
      def    __init__(self,    latent_dim=20,    hidden_dim=400,
output_dim=784):
        super().__init__()
        self.fc1 = nn.Linear(latent_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, z):
        h = F.relu(self.fc1(z))
        return torch.sigmoid(self.fc2(h))

def reparameterize(mu, logvar):
    std = torch.exp(0.5 * logvar)
    eps = torch.randn_like(std)
    return mu + eps * std

# Exemplo de forward
encoder = Encoder()
decoder = Decoder()
x = torch.randn((64, 784))  # batch de imagens MNIST achatadas
mu, logvar = encoder(x)
z = reparameterize(mu, logvar)
x_recon = decoder(z)
```

Código-fonte 1 - Variational Autoencoder em Pytorch
Fonte: Elaborado pelo autor (2025)

Os VAEs representam um passo decisivo em direção ao aprendizado generativo verdadeiramente probabilístico. Sua elegância matemática, aliada à flexibilidade das redes neurais profundas, permite que esses modelos aprendam

representações úteis, robustas e interpretáveis dos dados. Apesar de não serem campeões em qualidade visual, seu potencial como ferramentas para compressão, interpolação, análise de similaridade, detecção de anomalias e geração controlada é vasto e consolidado. Na próxima seção, conheceremos uma outra família de modelos generativos ainda mais impactante em termos visuais: as Redes Geradoras Adversariais (GANs).

FIAP

4 REDES GERADORAS ADVERSARIAIS (GANS)

Enquanto os VAEs são conhecidos por sua elegância matemática e estabilidade durante o treinamento, os Generative Adversarial Networks (GANs) revolucionaram o campo dos modelos generativos com sua impressionante capacidade de produzir amostras realistas. Introduzidas por Ian Goodfellow e colegas em 2014, as GANs são conhecidas por gerarem imagens, vídeos e até músicas com qualidade que, em muitos casos, até enganam os olhos e ouvidos humanos. Sua aplicação vai desde a criação de rostos falsos em alta resolução até a simulação de ambientes em jogos, restauração de fotos antigas e aumento de dados em tarefas de visão computacional.

4.1 Intuição das GANs: Duelo entre dois modelos

A intuição por trás das GANs é inspirada em um jogo entre dois jogadores, que aprendem simultaneamente, mas com objetivos opostos. Por um lado, temos o **gerador** (generator), que tenta produzir dados falsos que se pareçam com os dados reais. Por outro lado, há o **discriminador** (discriminator), cujo objetivo é distinguir entre dados reais (vindos do conjunto de treinamento) e dados falsos (gerados pelo gerador).

O gerador é, portanto, um falsificador. O discriminador é o fiscal. O processo pode ser comparado a uma falsificação de moeda: o falsificador tenta produzir cédulas cada vez mais parecidas com as verdadeiras, enquanto o fiscal tenta detectar quais são falsas. Com o tempo, se ambos forem igualmente competentes, o falsificador criará cédulas quase perfeitas, indistinguíveis das verdadeiras.

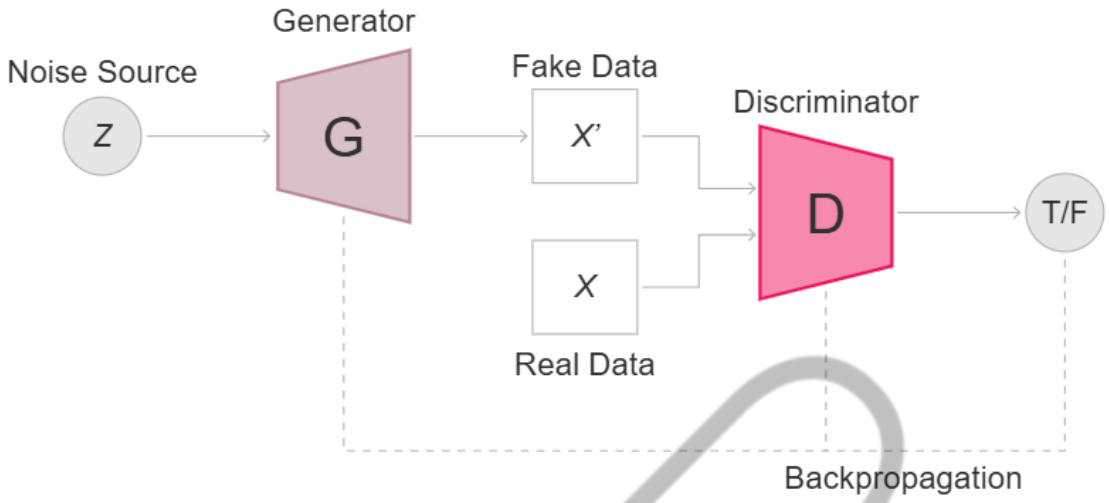


Figura 9 - Arquitetura completa de uma GAN
Fonte: ResearchGate (2019), adaptado por FIAP (2026)

4.2 Gerador *versus* discriminador

Formalmente, o gerador é uma rede neural que recebe como entrada um vetor de ruído aleatório $z \sim N(0, I)$ e produz uma amostra sintética $G(z)$, como uma imagem de um rosto, por exemplo. Já o discriminador recebe uma imagem (real ou gerada) e retorna a probabilidade de que ela seja real.

Durante o treinamento, o gerador tenta “enganar” o discriminador, e o discriminador tenta identificar corretamente o que é real e o que é gerado. Ambos são treinados alternadamente, com o discriminador aprendendo a classificar imagens e o gerador tentando maximizar a chance de suas amostras serem classificadas como reais.

O equilíbrio entre essas redes define o sucesso do modelo. Se o discriminador for muito bom, o gerador não consegue aprender. Se o gerador for muito eficiente desde o início, o discriminador não aprende a detectar padrões. A dinâmica adversarial é o que torna o treinamento poderoso, mas também instável.

4.3 Função objetivo: Minimax Game

A função de perda das GANs é baseada em uma formulação minimax, onde o gerador quer minimizar a probabilidade de ser detectado, e o discriminador quer maximizar sua acurácia:

$$\min_{G} \max_{\mathrm{D}} E_{\{x \sim p_{dados}(x)\}} [\log D(x)] + E_{\{z \sim p_{z(z)}\}} [\log (1 - D(G(z)))]$$

Essa equação representa o jogo entre os dois jogadores. O discriminador D tenta maximizar o logaritmo da probabilidade de acertar quando a entrada é real ($D(x)$) e minimizar quando a entrada é falsa ($D(G(z))$). Já o gerador G tenta minimizar o logaritmo da probabilidade de o discriminador detectar que suas imagens são falsas.

4.4 Teoria Matemática: Divergência de Jensen-Shannon

O treinamento ideal de uma GAN corresponde, teoricamente, à minimização da divergência de Jensen-Shannon (JS) entre a distribuição real dos dados p_{dados} e a distribuição gerada p_G. Essa divergência mede o quanto as duas distribuições de probabilidade são diferentes.

No ponto ótimo do treinamento, o gerador aprendeu tão bem a imitar a distribuição real que o discriminador não consegue mais distinguir entre real e falso, e sua predição se estabiliza em 0.5 para ambas as entradas. Nesse momento, as distribuições são indistinguíveis: a divergência JS é mínima e a GAN está em equilíbrio.

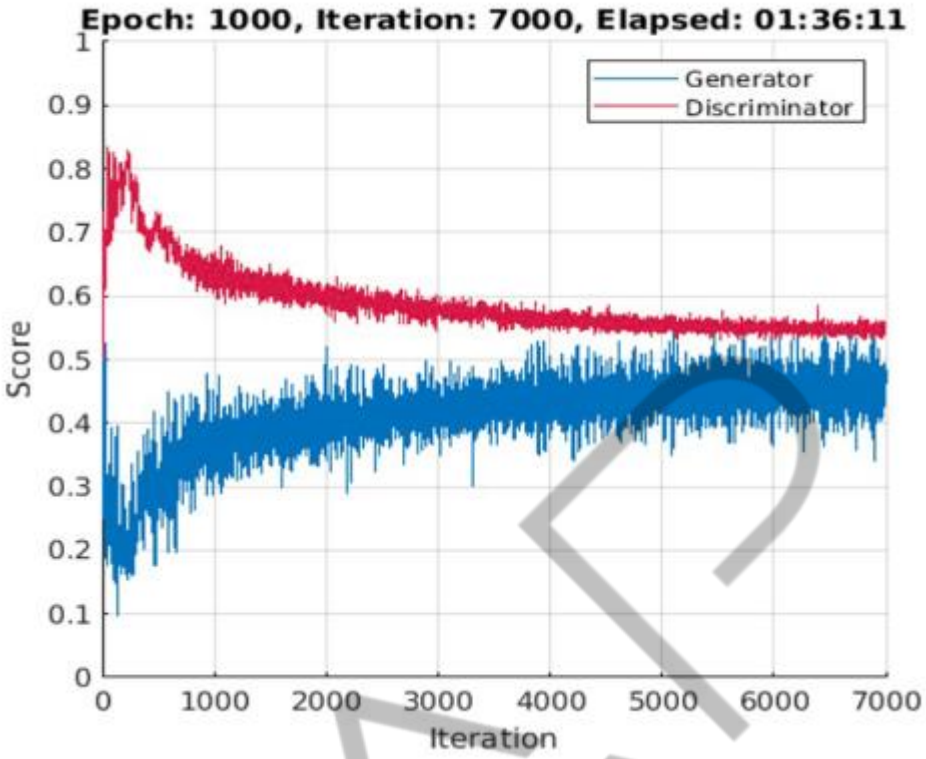


Figura 10 - Convergência da GAN
Fonte: Matlab (2025), adaptado por FIAP (2025)

4.5 Por que as GANs são difíceis de treinar?

Apesar da teoria elegante, o treinamento de GANs é notoriamente instável. Um dos principais problemas é o colapso de modo (*mode collapse*): quando o gerador aprende a produzir apenas um ou poucos tipos de amostras, ignorando a diversidade presente nos dados reais. Isso acontece quando o gerador encontra uma estratégia capaz de enganar o discriminador, mas não representa bem toda a distribuição de dados.

Outro problema é a dinâmica oscilatória durante o treinamento. Como os objetivos do gerador e do discriminador estão em oposição, é comum que a função de perda não convirja, mas fique oscilando indefinidamente.

Além disso, se o discriminador ficar muito bom muito rápido, ele começa a retornar gradientes quase nulos ao gerador, dificultando o aprendizado. Esse problema é conhecido como desvanecimento do gradiente (*vanishing gradients*).

Por essas razões, surgiram várias melhorias nas GANs clássicas, como a Wasserstein GAN (WGAN), que utiliza uma métrica diferente de divergência (distância de Wasserstein) e melhora significativamente a estabilidade.

4.6 Exemplos visuais: Real *versus* Fake e caminhada latente

Durante o treinamento de uma GAN, é possível visualizar o progresso observando as amostras geradas em cada época. No início, elas são apenas ruído. Com o tempo, começam a surgir formas, padrões e imagens reconhecíveis. Também é interessante realizar uma **caminhada no espaço latente** (*latent walk*), onde interpolamos vetores zzz e observamos como a saída do gerador muda de maneira suave.

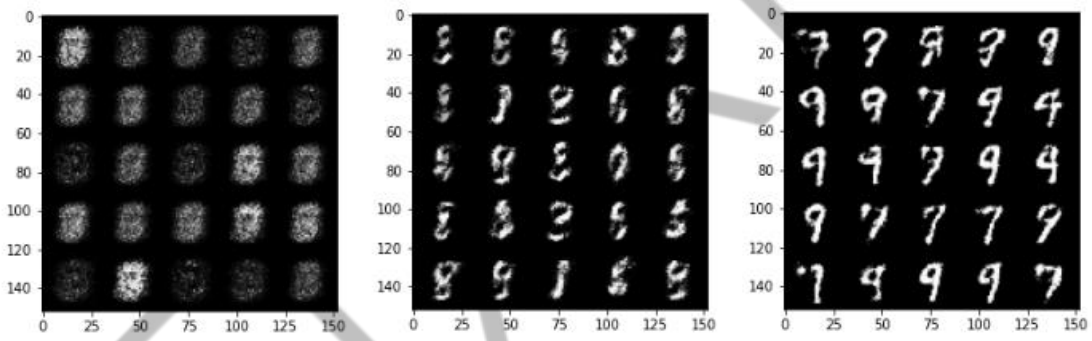


Figura 11 - Sequência de imagens geradas durante o treinamento da GAN
Fonte: Medium (2021)

4.7 Exemplo de código (PyTorch): DCGAN com CIFAR-10

A seguir está um exemplo básico de como implementar uma GAN com convoluções (DCGAN) para gerar imagens do dataset CIFAR-10:

```
import torch
import torch.nn as nn

# Gerador
class Generator(nn.Module):
    def __init__(self, latent_dim=100, img_channels=3):
        super().__init__()
        self.main = nn.Sequential(
                nn.ConvTranspose2d(latent_dim, 128, 4, 1, 0,
bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(True),
            nn.ConvTranspose2d(128, 64, 4, 2, 1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(True),
                nn.ConvTranspose2d(64, img_channels, 4, 2, 1,
bias=False),
            nn.Tanh()
        )

    def forward(self, input):
        return self.main(input)

# Discriminador
class Discriminator(nn.Module):
    def __init__(self, img_channels=3):
        super().__init__()
        self.main = nn.Sequential(
            nn.Conv2d(img_channels, 64, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(64, 128, 4, 2, 1, bias=False),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Flatten(),
            nn.Linear(128*8*8, 1),
            nn.Sigmoid()
        )

    def forward(self, input):
        return self.main(input).view(-1)
```

Código-fonte 2 - Deep Convolutional GAN em PyTorch
Fonte: Elaborado pelo autor (2025)

Este código define as duas redes principais. O gerador usa camadas de transposição convolucional para transformar um vetor latente z em uma imagem 3x32x32. E o discriminador usa convoluções para reduzir a imagem a uma decisão binária (real ou falso).

O treinamento alterna entre atualizar o discriminador com exemplos reais e gerados, e atualizar o gerador com base no erro do discriminador.

4.8 Dicas de treinamento para GANs

Treinar GANs exige alguns cuidados importantes. Usar BatchNorm e LeakyReLU ajuda na estabilidade das ativações. A função de ativação Tanh na saída do gerador e a normalização dos dados para a faixa [−1,1] também são práticas recomendadas.

A escolha do otimizador e da taxa de aprendizado pode ter grande impacto. O otimizador Adam, com parâmetros β1=0.5 e β2=0.999, tem mostrado bons resultados e pode ser um bom ponto de partida.

Evitar que o discriminador fique muito à frente do gerador é fundamental. Por isso, em algumas implementações, o discriminador é treinado uma ou duas vezes mais que o gerador, mas com cuidado para evitar que a competição fique desbalanceada.

4.9 Conclusão

As GANs representam uma das contribuições mais empolgantes da última década para a Inteligência Artificial. Seu princípio de competição leva a resultados altamente realistas e sua capacidade de geração de dados sintéticos é inigualável. No entanto, sua potência vem acompanhada de desafios técnicos significativos. Entender sua arquitetura, as dificuldades no treinamento e as estratégias para mitigar problemas como colapso de modo e instabilidade é essencial para quem deseja aplicá-las com sucesso. Na próxima seção, exploraremos as diversas variantes e aprimoramentos das GANs, como DCGAN, CycleGAN e StyleGAN, que ampliaram ainda mais o alcance dessa tecnologia.

5 EXTENSÕES E VARIAÇÕES DAS GANS

Desde a criação das GANs originais por Ian Goodfellow e sua equipe em 2014, uma enxurrada de variações e melhorias surgiram para enfrentar os desafios do treinamento, aumentar a qualidade das amostras geradas e expandir o uso das GANs para novos domínios. Entre as variações mais influentes estão a DCGAN, a CycleGAN e a StyleGAN. Cada uma dessas arquiteturas introduziu inovações importantes que se tornaram referências obrigatórias no campo da Inteligência Artificial Generativa.

Neste tópico, vamos explorar essas três variantes em detalhes, compreendendo como elas foram concebidas, o que as torna especiais e como elas transformaram a forma como geramos imagens com redes neurais.

5.1 DCGAN: Redes Convolucionais como Base para GANs Estáveis

A DCGAN (Deep Convolutional GAN) foi proposta por Alec Radford, Luke Metz e Soumith Chintala em 2015 como uma tentativa de tornar o treinamento de GANs mais estável e adequado para domínios visuais. A ideia principal foi substituir as camadas totalmente conectadas tradicionais por camadas convolucionais profundas no discriminador e por camadas de transposição convolucional no gerador.

Essas mudanças permitiram que os modelos extraíssem e reconstruíssem características espaciais das imagens com muito mais eficácia, especialmente em domínios visuais como CIFAR-10 e LSUN. Além disso, a introdução do uso de Batch Normalization nas camadas internas e ReLU no gerador (exceto na saída) e LeakyReLU no discriminador, ajudou a mitigar problemas como saturação de gradientes e instabilidade na convergência.

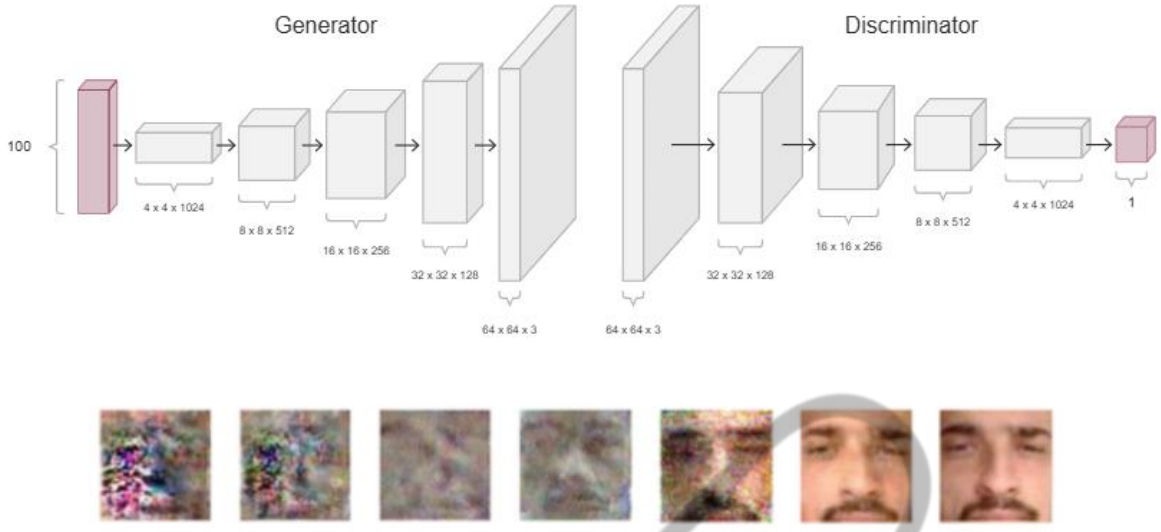


Figura 12 - Deep Convolution GAN
Fonte: ResearchGate (2024), adaptado por FIAP (2026)

Na prática, a DCGAN foi uma das primeiras variantes a mostrar que as GANs podiam gerar rostos, interiores de casas, carros e objetos reconhecíveis, mesmo com conjuntos de dados relativamente simples. A estabilidade no treinamento e a qualidade visual tornaram a DCGAN um modelo base para experimentos e aplicações até hoje.

5.2 CycleGAN: Tradução de Domínios sem Pares (Unpaired Image-to-Image Translation)

A CycleGAN, introduzida por Jun-Yan Zhu, Taesung Park, Phillip Isola e Alexei Efros em 2017, solucionou um problema crítico: como traduzir uma imagem de um domínio para outro sem ter exemplos pareados? Em muitos cenários do mundo real, como transformar uma pintura no estilo Van Gogh em uma fotografia ou transformar imagens de cavalos em zebras, não temos dados diretamente pareados (ou seja, uma imagem de um cavalo e a correspondente imagem da mesma cena como uma zebra).

A CycleGAN introduz o conceito de consistência cíclica. A ideia é treinar dois geradores: um que mapeia do domínio A para o domínio B (por exemplo, cavalo → zebra) e outro que faz o caminho inverso (zebra → cavalo). Ao aplicar ambos os geradores em sequência, o modelo deve ser capaz de retornar à imagem original, garantindo que a transformação preserve a estrutura essencial da imagem.

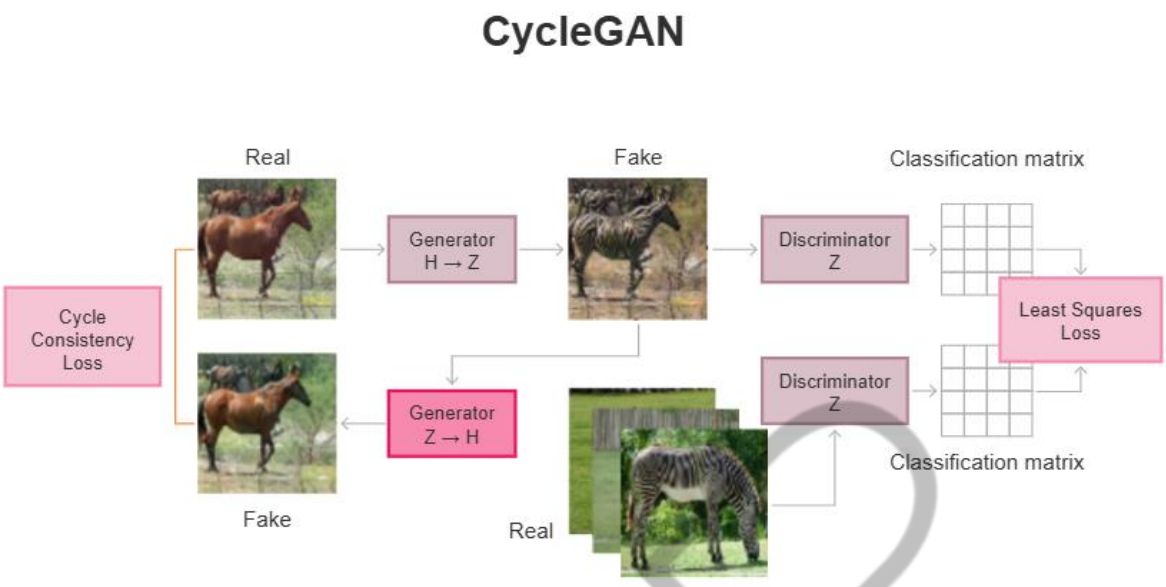


Figura 13 - CycleGAN.
Fonte: Medium (2024), adaptado por FIAP (2026)

O treinamento da CycleGAN é realizado com duas funções de perda principais: a perda adversarial, que assegura que as imagens traduzidas pareçam reais no domínio de destino, e a perda cíclica, que penaliza a diferença entre a imagem original e a reconstruída.

A aplicação da CycleGAN se estende a tarefas como restauração de fotografias antigas, transformação de imagens de satélite em mapas de rua, tradução de estilos artísticos e muito mais. Sua arquitetura e ideia de ciclo influenciaram uma geração de modelos de tradução de domínios.

5.3 StyleGAN: Geração de Rostos Realistas e Controle de Estilo

A StyleGAN é, provavelmente, a variante mais famosa e impressionante das GANs. Desenvolvida por Tero Karras, Samuli Laine e Timo Aila no laboratório de pesquisa da NVIDIA, a StyleGAN foi introduzida em 2018 e causou enorme impacto devido à sua capacidade de gerar rostos humanos altamente realistas, completamente sintéticos, mas indistinguíveis dos reais.

O principal diferencial da StyleGAN está na sua arquitetura de geração. Em vez de receber um vetor z direto e propagá-lo por todo o gerador como nas GANs tradicionais, a StyleGAN introduz uma rede de mapeamento separada que transforma z em um vetor intermediário w, e usa esse vetor para modular cada camada do

gerador por meio de operações de AdaIN (Adaptive Instance Normalization). Isso permite um controle fino sobre o estilo e os atributos da imagem gerada, como forma do rosto, cabelo, idade e expressão.

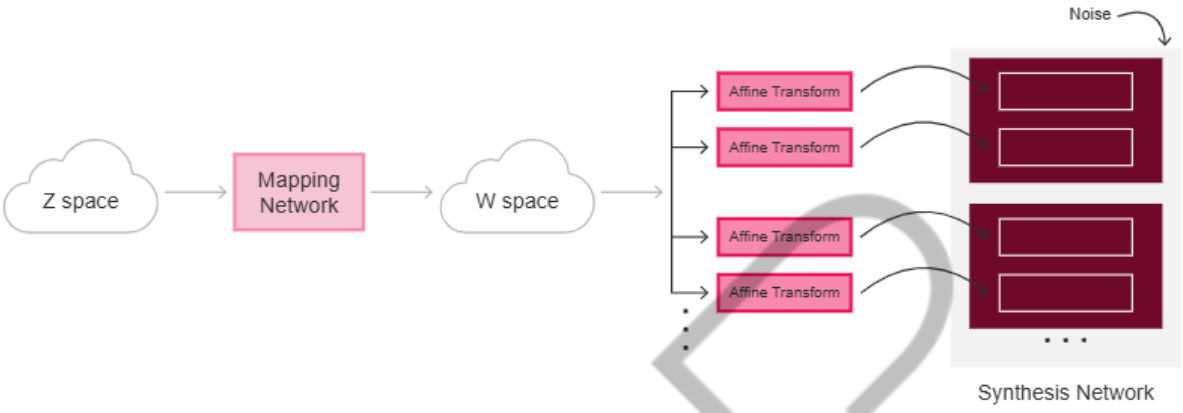


Figura 14 - StyleGAN
Fonte: Medium (2022), adaptado por FIAP (2026)

Com a StyleGAN, tornou-se possível não só gerar rostos realistas, mas também manipular com precisão características específicas. Por exemplo, ao alterar seletivamente o vetor latente em camadas mais profundas, o modelo pode mudar o tipo de cabelo de uma pessoa sem alterar sua identidade facial. Além disso, a interpolação entre dois rostos produz transições incrivelmente suaves e realistas.

A StyleGAN foi utilizada como base para projetos como ThisPersonDoesNotExist.com, onde cada atualização da página gera o rosto de uma pessoa que nunca existiu.

Com o tempo, a StyleGAN evoluiu para a StyleGAN2 e StyleGAN3, que aprimoraram a fidelidade visual, corrigiram artefatos e introduziram melhor controle de geometria e coerência espacial.

5.4 Comparação entre DCGAN, CycleGAN e StyleGAN

Para visualizar melhor as diferenças entre essas variantes, podemos apresentar um quadro comparativo com seus principais atributos:

Modelo	Principal Inovação	Tipo de Aplicação	Controle Latente	Necessita Pares?	Exemplo Típico
DCGAN	Uso de convoluções profundas.	Geração de imagens simples.	Baixo.	Não.	CIFAR-10, rostos básicos.
CycleGAN	Tradução cíclica sem imagens pareadas.	Tradução de domínios	Médio.	Não.	Cavalos ↔ Zebras, Van Gogh ↔ Fotos.
StyleGAN	Separação do estilo e controle fino.	Rostos realistas, retratos.	Alto.	Não.	Geração de pessoas fictícias.

Quadro 1 - Comparação entre DCGAN, CycleGAN e StyleGAN
Fonte: Elaborado pelo autor (2025)

Cada modelo representa uma evolução no que se entende por geração realista e controlável. A DCGAN deu estabilidade e forma inicial às GANs para domínios visuais, a CycleGAN ampliou sua aplicação para tarefas de tradução e a StyleGAN levou a fidelidade visual ao seu ápice, possibilitando até mesmo editores de rosto com IA.

5.5 Conclusão

As extensões das GANs mostram como a comunidade de pesquisa tem conseguido superar limitações técnicas e adaptar a ideia original de adversariedade a uma variedade impressionante de aplicações. Hoje, GANs não são apenas modelos para gerar imagens, mas ferramentas sofisticadas para transformação de domínios, criação de conteúdo sintético, manipulação de atributos e até arte interativa.

Dominar as variantes como a DCGAN, a CycleGAN e a StyleGAN é fundamental para compreender o potencial atual dos modelos generativos e para explorar suas possibilidades futuras — seja no cinema, na moda, na medicina ou na robótica. A seguir, vamos colocar lado a lado os dois grandes paradigmas que estudamos até aqui, VAEs e GANs, para entender suas diferenças, vantagens e casos de uso ideais.

6 VAES *VERSUS* GANS: UMA ANÁLISE COMPARATIVA

Após estudarmos em profundidade os Autoencoders Variacionais (VAEs) e as Redes Geradoras Adversariais (GANs), é natural que surja a pergunta: qual modelo é o melhor para tarefas de geração de dados? A resposta, como muitas vezes ocorre em ciência de dados, é: depende do objetivo da aplicação. Ambos os modelos representam abordagens poderosas, mas com fundamentos teóricos, arquiteturas e comportamentos muito diferentes.

Nesta seção, vamos analisar comparativamente os principais aspectos que os distinguem, desde a qualidade das saídas até a estrutura do espaço latente, passando também pela estabilidade no treinamento e pelos casos de uso mais adequados.

6.1 Qualidade das Saídas: Realismo *versus* Coerência Estatística

Um dos critérios mais frequentemente utilizados para comparar modelos generativos é a qualidade visual (ou perceptiva) das amostras produzidas. Nesse aspecto, as GANs são claramente superiores. Como a sua função não busca apenas reconstrução fiel, mas sim enganar um discriminador treinado para reconhecer dados reais, as GANs tendem a gerar amostras mais nítidas, detalhadas e realistas.

Por exemplo, ao treinar um VAE e uma GAN para gerar rostos humanos, o VAE frequentemente resultará em imagens ligeiramente borradas, como se estivessem com um leve desfoque. Isso ocorre porque o VAE maximiza a probabilidade de reconstrução sob uma distribuição gaussiana, o que o leva a “suavizar” regiões incertas em vez de produzir detalhes nítidos.

Já a GAN, ao competir contra o discriminador, é pressionada a produzir imagens com bordas nítidas, contrastes bem definidos e padrões realistas, mesmo que, às vezes, essas imagens não respeitem totalmente a diversidade estatística dos dados originais.



Figura 15 - Imagens geradas usando VAE (acima) e GAN (abaixo)
Fonte: ResearchGate (2016)

6.2 Estrutura do Espaço Latente: Regularidade *versus* Complexidade

Outro ponto importante é a estrutura do espaço latente, ou seja, como o modelo organiza internamente os dados em uma representação comprimida que pode ser usada para gerar novos exemplos.

Os VAEs se destacam nesse quesito. Como seu espaço latente é explicitamente regularizado para seguir uma distribuição normal multivariada, ele é contínuo, navegável e *interpretable*. Isso significa que é possível interpolar de forma suave entre dois pontos latentes e gerar transições coerentes. Também é possível medir distâncias no espaço latente para avaliar a similaridade entre exemplos.

Por exemplo, em um VAE treinado com dígitos manuscritos, é possível gerar uma transição suave do número 3 para o número 8, explorando o espaço latente entre os dois vetores z. Essa navegabilidade torna os VAEs ideais para tarefas que exigem entendimento semântico e controle sobre atributos.

Já as GANs não impõem uma estrutura explícita ao espaço latente. Embora ainda seja possível interpolar entre vetores de entrada z e obter transições interessantes, o comportamento do gerador nesse espaço é mais irregular e difícil de interpretar. Isso pode ser parcialmente resolvido em variantes como a StyleGAN, que introduz mapeamentos mais controlados no espaço latente, mas na GAN clássica o controle é mais limitado.

6.3 Estabilidade no Treinamento: Convergência *versus* Instabilidade

No que diz respeito à estabilidade do treinamento, os VAEs novamente levam vantagem. Como são baseados em uma função de perda bem definida, derivada da teoria da inferência variacional, os VAEs convergem de maneira estável em praticamente todos os contextos. O principal desafio está em ajustar corretamente a relação entre o termo de reconstrução e a regularização pela divergência KL.

Em contrapartida, as GANs são notoriamente difíceis de treinar. Como envolvem dois modelos em competição — gerador e discriminador —, o processo pode entrar em oscilações, colapsos e instabilidades. Problemas como colapso de modo, onde o gerador aprende a gerar apenas um ou poucos tipos de amostra, são comuns. Além disso, o balanceamento entre os dois jogadores exige um cuidado considerável: se o discriminador se tornar muito forte, o gerador deixa de receber gradientes úteis para aprender.

No entanto, a literatura propõe diversas estratégias para mitigar esses problemas, como o uso de Wasserstein GANs, normalizações específicas, penalizações de gradiente e ajustes finos na taxa de aprendizado. Ainda assim, o treinamento de GANs é muitas vezes descrito como “uma arte tanto quanto uma ciência”.

6.4 Comparação de Aplicações: Quando usar cada um

O uso ideal de VAEs ou GANs depende fortemente do objetivo da aplicação. Quando a prioridade é interpretabilidade, compactação de dados, interpolação semântica ou detecção de anomalias, os VAEs são mais apropriados. Por exemplo, em sistemas de recomendação, é comum usar VAEs para gerar representações latentes compactas de usuários e produtos, com ótimo desempenho em tarefas de recuperação de informação.

Por outro lado, quando a demanda é por qualidade visual extrema, realismo de imagem ou criação de conteúdo artístico, as GANs são claramente superiores. Aplicações como deepfakes, geração de avatares, melhoria de resolução (super-

resolution) e arte digital dependem da capacidade das GANs de produzir amostras de alta fidelidade.

Além disso, GANs são amplamente utilizadas para a aumentação de dados, especialmente em contextos onde o volume de dados reais é escasso, como no caso de imagens médicas, pois suas amostras são convincentes o suficiente para treinar classificadores robustos.

Podemos sintetizar os principais pontos desta comparação em um quadro resumo:

Critério	VAE	GAN
Qualidade das amostras.	Média, com tendência a borrões.	Alta, com detalhes realistas.
Estrutura do espaço latente.	Contínuo, regular, interpretável.	Irregular, menos controlável.
Estabilidade no treinamento.	Alta (convergência garantida).	Baixa (oscilações, colapso de modo).
Amostragem direta.	Sim, por distribuição conhecida.	Sim, mas com controle limitado.
Aplicações típicas.	Compressão, interpolação, análise semântica.	Criação artística, aumento de dados, deepfakes.
Complexidade matemática.	Moderada, baseada em inferência variacional.	Alta, com otimização adversarial.

Quadro 2 - Comparação entre VAE e GAN
Fonte: Elaborado pelo autor (2025)

Esse quadro ajudará a guiar decisões de projeto. Em muitos sistemas práticos, é comum combinar VAEs e GANs em uma mesma arquitetura, como nos VAE-GANs, que tentam reunir o melhor dos dois mundos: a estrutura semântica dos VAEs e o realismo visual das GANs.

6.5 Conclusão

VAEs e GANs não são modelos concorrentes, mas sim abordagens complementares dentro do rico ecossistema dos modelos generativos. Enquanto os

VAEs oferecem um arcabouço probabilístico rigoroso e interpretável, as GANs brilham no quesito expressividade visual. Dominar as diferenças entre eles permite escolher a melhor ferramenta para cada problema, ou até mesmo combiná-las para ir além das limitações de cada abordagem.

Na próxima seção, vamos sair do campo da teoria e mergulhar na prática. Será apresentado um tutorial completo de implementação, passo a passo, com exemplos reais de código e visualizações, para consolidar todo o conhecimento adquirido até aqui.

FIAP

7 TUTORIAL PRÁTICO: IMPLEMENTANDO UM VAE E UMA GAN COM MNIST

Após uma jornada teórica e conceitual pelos autoencoders variacionais e pelas redes geradoras adversariais, é hora de colocar a mão na massa. Esta seção oferece um tutorial prático e didático, utilizando Python com as bibliotecas PyTorch e Keras, dois dos mais populares frameworks de Deep Learning.

A ideia é treinar um VAE e uma GAN com o dataset MNIST, um clássico da visão computacional que contém imagens de dígitos manuscritos (0 a 9) com resolução de 28x28 pixels em escala de cinza. Para variação, o leitor também pode testar o FashionMNIST, que segue a mesma estrutura, mas contém imagens de roupas, uma escolha excelente para testar a generalização visual.

7.1 Preparação: Imports e Dataset

Antes de construir os modelos, é preciso importar as bibliotecas e carregar os dados. Vamos usar PyTorch neste exemplo, mas o código pode ser facilmente adaptado para Keras.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import matplotlib.pyplot as plt

# Transformações para normalização
transform = transforms.Compose([
    transforms.ToTensor(),
])

# Dataset MNIST
train_dataset = datasets.MNIST(root='./data', train=True,
download=True, transform=transform)
train_loader = DataLoader(train_dataset, batch_size=128,
shuffle=True)
```

Código-fonte 3 - Importando bibliotecas e baixando os dados do MNIST
Fonte: Elaborado pelo autor (2025)

O dataset é baixado automaticamente e carregado com normalização dos pixels para o intervalo [-1, 1], o que facilita o treinamento dos modelos.

7.2 VAE: Construindo o encoder e o decoder

Vamos começar com o VAE, cuja arquitetura é composta por um encoder que mapeia a imagem para dois vetores (média e variância logarítmica), uma etapa de reparametrização, e um decoder que reconstrói a imagem.

```
class VAE(nn.Module):
    def __init__(self, latent_dim=20):
        super().__init__()
        self.fc1 = nn.Linear(28*28, 400)
        self.fc21 = nn.Linear(400, latent_dim)    # média
        self.fc22 = nn.Linear(400, latent_dim)    # logvar
        self.fc3 = nn.Linear(latent_dim, 400)
        self.fc4 = nn.Linear(400, 28*28)

    def encode(self, x):
        h1 = F.relu(self.fc1(x))
        return self.fc21(h1), self.fc22(h1)  # média e logvar

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5*logvar)
        eps = torch.randn_like(std)
        return mu + eps * std

    def decode(self, z):
        h3 = F.relu(self.fc3(z))
        return torch.sigmoid(self.fc4(h3))

    def forward(self, x):
        mu, logvar = self.encode(x.view(-1, 28*28))
        z = self.reparameterize(mu, logvar)
        recon = self.decode(z)
        return recon, mu, logvar
```

Código-fonte 4 - Construindo o Encoder-Decoder
Fonte: Elaborado pelo autor (2025)

Cada bloco é simples, porém poderoso. A reparametrização permite o fluxo do gradiente através da amostragem. A saída é reconstruída com uma função de ativação sigmoid, pois os pixels estão normalizados entre 0 e 1.

7.2.1 Função de perda e treinamento do VAE

A função de perda combina o erro de reconstrução (cross-entropy ou MSE) com a divergência KL:

```
def vae_loss(recon_x, x, mu, logvar):
    BCE = F.binary_cross_entropy(recon_x, x.view(-1, 28*28), reduction='sum')
    KLD = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return BCE + KLD
```

Código-fonte 5 - Função de Custo
Fonte: Elaborado pelo autor (2025)

O treinamento segue a rotina padrão do PyTorch:

```
vae = VAE()
optimizer = torch.optim.Adam(vae.parameters(), lr=1e-3)
vae.train()

for epoch in range(10):
    total_loss = 0
    for x, _ in train_loader:
        optimizer.zero_grad()
        recon, mu, logvar = vae(x)
        loss = vae_loss(recon, x, mu, logvar)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch+1}, Loss: {total_loss / len(train_loader.dataset):.4f}")
```

Código-fonte 6 - Executando o treinamento.
Fonte: Elaborado pelo autor (2025)

7.2.2 Gerando novas amostras com o VAE

Após o treinamento, podemos amostrar do espaço latente padrão $N(0, I)$ para gerar novas imagens:

```
vae.eval()
with torch.no_grad():
    z = torch.randn(64, 20)  # 64 amostras
    samples = vae.decode(z).view(-1, 1, 28, 28)

# Visualização
grid = torch.cat([s for s in samples])
plt.imshow(torchvision.utils.make_grid(grid, nrow=8).permute(1,
2, 0))
plt.title("Amostras Geradas - VAE")
plt.axis('off')
plt.show()
```

Código-fonte 7 - Gerando novas imagens com o VAE treinado
Fonte: Elaborado pelo autor (2025)



Figura 16 - Imagens geradas utilizando VAE
Fonte: Elaborado pelo autor (2025)

7.2.3 Explorando o espaço latente com o VAE

Uma maneira interessante de analisar a qualidade do VAE é realizar uma interpolação entre dois vetores latentes:

```
z1 = torch.randn(1, 20)
z2 = torch.randn(1, 20)
alphas = torch.linspace(0, 1, steps=10)

interpolated = [(1 - alpha) * z1 + alpha * z2 for alpha in
alphas]
interpolated = torch.cat(interpolated)

with torch.no_grad():
    images = vae.decode(interpolated).view(-1, 1, 28, 28)

# Visualização
grid = torchvision.utils.make_grid(images,
nrow=10).permute(1, 2, 0).squeeze(-1)
plt.imshow(grid, cmap='gray')
plt.title("Interpolação no Espaço Latente")
plt.axis('off')
plt.show()
```

Código-fonte 8 - Importando bibliotecas e baixando os dados do MNIST
Fonte: Elaborado pelo autor (2025)

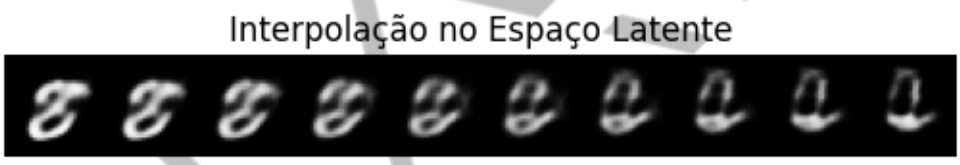


Figura 17 - Sequência de imagens interpoladas entre dois dígitos, mostrando transição visual suave
Fonte: Elaborado pelo autor (2025)

7.3 Treinando uma GAN simples com MNIST

Agora, partimos para a GAN. Começamos com o gerador, que transforma um vetor de ruído em uma imagem, e o discriminador, que decide se uma imagem é real ou falsa:

```
class Generator(nn.Module):
    def __init__(self, latent_dim=100):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(latent_dim, 256),
            nn.ReLU(True),
            nn.Linear(256, 512),
            nn.ReLU(True),
            nn.Linear(512, 784),
            nn.Tanh()
        )

def forward(self, z):
```

```
        return self.model(z).view(-1, 1, 28, 28)

class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Flatten(),
            nn.Linear(784, 512),
            nn.LeakyReLU(0.2),
            nn.Linear(512, 256),
            nn.LeakyReLU(0.2),
            nn.Linear(256, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.model(x)
```

Código-fonte 9 - Importando bibliotecas e baixando os dados do MNIST
Fonte: Elaborado pelo autor (2025)

O gerador termina com tanh, e os dados reais devem estar na mesma faixa [−1,1]. Ajuste a normalização do dataset conforme necessário.

7.3.1 Treinamento da GAN

A função de perda utilizada é a binary cross entropy. O treinamento alterna entre atualizar o discriminador e o gerador:

```
G = Generator()
D = Discriminator()

opt_G = torch.optim.Adam(G.parameters(), lr=0.0002)
opt_D = torch.optim.Adam(D.parameters(), lr=0.0002)

criterion = nn.BCELoss()

for epoch in range(20):
    for x_real, _ in train_loader:
        # Preparar dados
        real = x_real
        batch_size = real.size(0)
        labels_real = torch.ones(batch_size, 1)
        labels_fake = torch.zeros(batch_size, 1)

        # Treinar Discriminador
        z = torch.randn(batch_size, 100)
        fake = G(z)
```

```
            loss_D = criterion(D(real), labels_real) +
criterion(D(fake.detach()), labels_fake)
        opt_D.zero_grad()
        loss_D.backward()
        opt_D.step()

        # Treinar Gerador
        z = torch.randn(batch_size, 100)
        fake = G(z)
        loss_G = criterion(D(fake), labels_real)
        opt_G.zero_grad()
        loss_G.backward()
        opt_G.step()

    print(f"Epoch {epoch+1}, Loss D: {loss_D.item():.4f}, Loss
G: {loss_G.item():.4f}")
```

Código-fonte 10 - Treinando uma GAN
Fonte: Elaborado pelo autor (2025)

7.3.2 Gerando imagens com a GAN

Após o treinamento, podemos gerar amostras passando vetores z aleatórios ao gerador:

```
G.eval()
with torch.no_grad():
    z = torch.randn(64, 100)
    fake_images = G(z)

# Visualização
grid = torchvision.utils.make_grid(fake_images, nrow=8,
normalize=True)
plt.imshow(grid.permute(1, 2, 0))
plt.title("Imagens Geradas - GAN")
plt.axis('off')
plt.show()
```

Código-fonte 11 - Gerando imagens com GAN
Fonte: Elaborado pelo autor (2025)

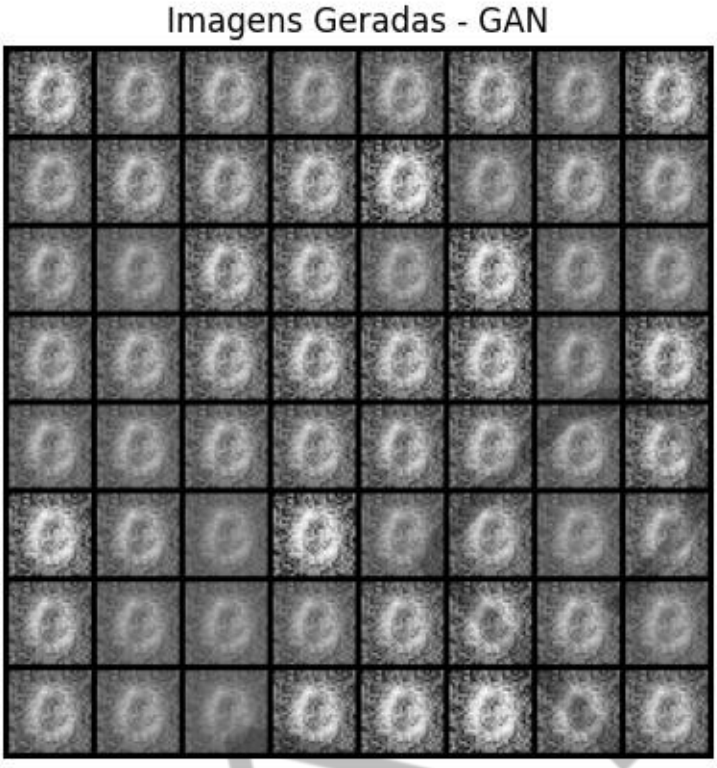


Figura 18 - Imagens geradas utilizando GAN
Fonte: Elaborado pelo autor (2025)

7.4 Dicas de avaliação

Para avaliar VAEs, recomenda-se visualizar o espaço latente, calcular a perda ELBO e medir a log-verossimilhança em dados de teste. Já para GANs, pode-se usar métricas como FID (Fréchet Inception Distance) e Inception Score, além da avaliação visual subjetiva.

A estabilidade do treinamento é um bom indicador da robustez do modelo. Recomenda-se sempre visualizar as amostras geradas a cada época para detectar possíveis colapsos de modo ou convergência ruim.

CONCLUSÃO

Neste capítulo, exploramos os principais fundamentos e aplicações dos modelos gerativos profundos, abordando desde conceitos básicos de Autoencoders até arquiteturas avançadas como Variational Autoencoders (VAEs) e Generative Adversarial Networks (GANs). Aprendemos como essas abordagens permitem criar, reconstruir e transformar dados de maneira realista, com aplicações que vão desde a síntese de imagens e geração de texto até a detecção de anomalias e aumento de dados para treinamento de outros modelos. Também discutimos as vantagens, as limitações e os desafios práticos, além de apresentar exemplos e implementações que demonstram o potencial e a complexidade desses modelos.

Ao final, ficou claro que VAEs e GANs representam duas abordagens complementares: enquanto os VAEs priorizam a estrutura e interpretabilidade do espaço latente, as GANs buscam realismo extremo nas amostras geradas. A compreensão desses métodos e suas variantes é essencial para avançar em aplicações modernas de Inteligência Artificial que demandam criatividade e geração de conteúdo.

FIAP

REFERÊNCIAS

ALMEIDA, Wesley. Construindo sua primeira GAN com PyTorch. **Medium**, 7 de fevereiro de 2021. Disponível em: <https://medium.com/turing-talks/construindo-sua-primeira-gan-com-pytorch-a47ddcc05649>. Acesso em: 29 ago. 2025.

ARIJZOUAOUI. StyleGAN: Explained. **Medium**, 27 de abril de 2022. Disponível em: <https://medium.com/@arijzouaoui/stylegan-explained-3297b4bb813a>. Acesso em: 29 ago. 2025.

AWS. What is a GAN? **AWS Amazon**, 2025. Disponível em: <https://aws-amazon-com.translate.goog/what-is/gan/?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=wa>. Acesso em: 29 ago. 2025.

BORGES, João Arthur de Oliveira. Detecção de Ataques de Botnets em IoT via Variational Autoencoder. **ResearchGate**, maio 2022. Disponível em: <https://www.researchgate.net/figure/Figura-1-Diagrama-que-demonstra-a-arquitetura-b-asica-de-um-Variational-Autoencoder_fig1_364401436>. Acesso em: 28 ago. 2025.

DATA604. GANs – Part10: CycleGAN. **Medium**, 26 de setembro de 2024. Disponível em: <https://medium.com/@Mustafa77/gans-part10-3bff433698ba>. Acesso em: 29 ago. 2025.

ESTEVES, Toni. Extraindo representações com autoencoders convolucionais, **Medium**, 29 de maio de 2020. Disponível em: <https://estevestoni.medium.com/extraindo-representa%C3%A7%C3%B5es-com-autoencoders-convolucionais-405ab73afa05>. Acesso em: 29 ago. 2025.

GAINETDINOV, Ainur. Diffusion Models vs. GANs vs. VAEs: Comparison of Deep Generative Models. **Medium**, 12 de maio de 2023. Disponível em: <https://pub.towardsai.net/diffusion-models-vs-gans-vs-vaes-comparison-of-deep-generative-models-67ab93e0d9ae>. Acesso em: 29 ago. 2025.

GENERATE Synthetic Signals Using Conditional GAN. **Matlab**, 2025. Disponível em: <https://www.mathworks.com/help/deeplearning/ug/generate-synthetic-pump-signals-using-conditional-generative-adversarial-network.html>. Acesso em: 29 ago. 2025.

GOODFELLOW, Ian J*. et al*. Generative adversarial nets. **Advances in neural information processing systems**, n. 27, 2014.

GOODFELLOW, I., BENGIO, Y., COURVILLE, A. **Deep learning**, 2016. MIT Press. Disponível em: <http://www.deeplearningbook.org>. Acesso em: 28 ago. 2025.

HARISH R. Denoising Autoencoders. **Medium**, 28 de julho de 2019. Disponível em: <https://medium.com/@harishr2301/denoising-autoencoders-996e866e5cd0>. Acesso em: 29 ago. 2025.

HASEGAWA-JOHNSON, Mark. Images generated by a VAE and a DCGAN. First row: samples from a VAE. Second row: samples from a DCGAN. **ResearchGate**, julho 2016. Disponível em: <https://www.researchgate.net/figure/mages-generated-by-a-VAE-and-a-DCGAN-First-row-samples-from-a-VAE-Second-row-samples_fig9_305654682>. Acesso em: 28 ago. 2025.

HOCHREITER S.; SCHMIDHUBER, J. Long Short-Term Memory. **Neural Computation**, v. 9, n. 8, p. 1735-1780, 15 Nov. 1997.

JAMIL, Faras. A schematic diagram of the multivariate deep autoencoder architecture. **ResearchGate**, setembro 2023. Disponível em: <https://www.researchgate.net/figure/A-schematic-diagram-of-the-multivariate-deep-autoencoder-architecture-The-input-layer_fig1_374279521>. Acesso em: 28 ago. 2025.

JAMIL, Faras *et al*. Wind turbine drivetrain fault detection using physics-informed multivariate deep learning. **ResearchGate**, setembro 2023. Disponível em: <https://www.researchgate.net/publication/372676809_Wind_turbine_drivetrain_fault_detection_using_physics-informed_multivariate_deep_learning>. Acesso em: 29 ago. 2025.

JENKINS, John; ROY, Kaushik. Exploring deep convolutional generative adversarial networks (DCGAN) in biometric systems: a survey study. **ResearchGate**, maio 2024. Disponível em: <https://www.researchgate.net/figure/llustration-of-DCGAN-framework-and-the-evolving-output-of-synthetic-face-images_fig2_380947622>. Acesso em: 28 ago. 2025.

LOUISET, Robin *et al*. UCSL : A Machine Learning Expectation-Maximization Framework for Unsupervised Clustering Driven by Supervised Learning. **Arxiv**, 5 de julho de 2021. Disponível em: <https://arxiv.org/abs/2107.01988>. Acesso em: 29 ago. 2025.

SHARVIT, Hadar. Why should we care about a DEEP network? **Medium**, 26 fev. 2024. Disponível em: <https://medium.com/@hadarsharvit/why-should-we-care-about-a-deep-network-0bcce5e39ff4>. Acesso em: 29 ago. 2025.

TONELLO, Andrea M. *et al*. Machine Learning Tips and Tricks for Power Line Communications. **IEEE**, v. 7, 17 de junho de 2019. Disponível em: <https://ieeexplore.ieee.org/document/8737766>. Acesso em: 28 ago. 2025.

VASWANI, Ashish *et al*. Attention is all you need. **Advances in neural information processing systems**, v. 30, 2017.

YEH, R. A. *et al.* Semantic Image Inpainting with Perceptual and Contextual Losses. **Arxiv**, 26 de julho de 2016. Disponível em: <https://arxiv.org/abs/1607.07539>. Acesso em: 29 ago. 2025.

FIAP